



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

MAESTRÍA EN INTELIGENCIA ARTIFICIAL

ASIGNATURA:

PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL

ACTIVIDAD:

PRESENTACIÓN DE LA FASE DE PREPARACION Y PROCESAMIENTO DE DATOS

AUTORES:

CABRERA PARRALES JOEL DAVID

SANCHEZ AREVALO ANIBAL ANDRES

TUTOR:

MG. ALEXANDRA J. ARCINIEGAS C.

SAMBORONDÓN – ECUADOR

ABRIL – 2026

1. Introducción

El presente informe documenta la fase de preparación y procesamiento de datos del proyecto de Análisis Comparativo de Técnicas de IA para Text-to-SQL con Visualización Automática, desarrollado como parte de la Maestría en Inteligencia Artificial de la Universidad de Especialidades Espíritu Santo (UEES).

La preparación de datos constituye una etapa crítica en cualquier proyecto de aprendizaje automático. Para sistemas Text-to-SQL, esta fase es particularmente exigente dado que los datos combinan dos modalidades: lenguaje natural (preguntas de usuario) y lenguaje estructurado (consultas SQL), cada una con sus propias características y desafíos de procesamiento.

1.1 Dataset Seleccionado

Se trabajó con el dataset Spider (Yale University, EMNLP 2018), descargado íntegramente desde Hugging Face Hub mediante la librería datasets. Spider es el benchmark estándar de la industria para la tarea Text-to-SQL, ampliamente citado en la literatura científica y adoptado como referencia en competencias internacionales.

Característica	Valor
Fuente	Yale University — EMNLP 2018
Hub	xlangai/spider (Hugging Face)
Total de ejemplos	8,034
Split Train	7,000 ejemplos
Split Validation	1,034 ejemplos
Bases de datos únicas	200
Dominios cubiertos	138
Licencia	CC BY-SA 4.0
Referencia	Yu et al. (2018), EMNLP

Cada ejemplo del dataset contiene: (1) una pregunta en lenguaje natural en inglés, (2) la consulta SQL correspondiente ejecutable sobre una base de datos relacional, (3) el identificador de la base de datos y (4) el nivel de dificultad de la consulta (easy, medium, hard, extra hard).

2. Resultados del Análisis Exploratorio de Datos (EDA)

El Análisis Exploratorio de Datos fue la primera etapa ejecutada sobre el dataset completo. Su objetivo fue caracterizar estadísticamente las variables, identificar problemas de calidad y guiar las decisiones de preprocesamiento posteriores.

2.1 Estadísticas Descriptivas Generales

A continuación se presentan las estadísticas descriptivas calculadas sobre las variables numéricas principales del dataset:

Variable	Media	Desv. Est.	Mínimo	Mediana	Máximo
Palabras en pregunta NL	12.69	4.31	1	12	74
Tokens en consulta SQL	17.8	11.2	3	14	168
Longitud SQL (chars)	~85	~55	12	70	820
Número de cláusulas SQL	3.1	1.4	1	3	7
Número de JOINS	0.82	0.94	0	1	6
Score de complejidad	2.63	2.91	0	2	23

2.2 Distribución de la Variable Objetivo: Dificultad SQL

Spider define cuatro niveles de dificultad para las consultas SQL. La distribución de estos niveles en el dataset completo (train + validation) es la siguiente:

Nivel	Cantidad	Porcentaje	Criterio SQL
Easy	2,380	29.6%	SELECT simple, sin JOIN ni GROUP BY
Medium	3,940	49.0%	Con JOIN o GROUP BY o ORDER BY
Hard	1,049	13.1%	Subconsultas o JOINS múltiples + GROUP BY
Extra Hard	665	8.3%	UNION / INTERSECT / EXCEPT / anidamiento complejo
TOTAL	8,034	100%	—

El ratio de desbalance entre la clase mayoritaria (medium: 49.0%) y la minoritaria (extra: 8.3%) es de 5.9x, lo que supera el umbral de 3x considerado problemático para modelos de clasificación.

2.3 Análisis de Cláusulas SQL por Nivel de Dificultad

Se analizó la presencia de las principales cláusulas SQL en función del nivel de dificultad, revelando patrones estructurales claros:

Cláusula SQL	Easy	Medium	Hard	Extra
JOIN	0%	72%	89%	78%
WHERE	68%	71%	82%	74%
GROUP BY	0%	51%	62%	45%
ORDER BY	0%	38%	29%	22%
HAVING	0%	0%	28%	31%
Subconsulta	0%	0%	62%	88%
UNION/INTERSECT	0%	0%	0%	41%

El análisis de cláusulas confirma la lógica de anotación original: la complejidad aumenta progresivamente con la incorporación de JOINS, subconsultas y operadores de conjunto.

2.4 Hallazgos del Análisis de Calidad de Datos

El reporte de calidad ejecutado sobre el dataset arrojó los siguientes resultados:

- Valores nulos: 0 en todas las columnas — el dataset no requiere imputación primaria.
- Duplicados exactos (pregunta + SQL idénticos): 21 registros — eliminados en limpieza.
- Preguntas duplicadas (misma pregunta, diferente SQL): 33 registros — corresponden a paráfrasis naturales.
- Consultas SQL duplicadas (mismo SQL, diferente pregunta): 1,043 — esperado y válido.
- Preguntas < 3 palabras: 0 — sin outliers extremos de longitud mínima.
- SQLs sin cláusula SELECT: 0 — todas las consultas están bien formadas.

La alta calidad base del dataset Spider refleja el proceso de anotación riguroso realizado por 11 estudiantes de Yale con revisión experta. Sin embargo, el desbalance de clases requiere tratamiento explícito.

3. Pipeline de Limpieza de Datos

Basándonos en los hallazgos del EDA, se diseñó e implementó un pipeline de limpieza sistemático de cuatro pasos. Cada paso está documentado y justificado académicamente para garantizar la reproducibilidad del proceso.

3.1 Pasos del Pipeline

Paso 1 — Normalización de texto en preguntas NL

Se aplicó normalización Unicode NFC para garantizar representación canónica de caracteres. Se eliminaron espacios múltiples y al inicio/fin de cada pregunta. Se estandarizó la capitalización (primera letra en mayúscula) y se aseguró que todas las preguntas terminen con signo de interrogación.

Justificación: la consistencia tipográfica reduce el vocabulario efectivo de los modelos de lenguaje y mejora la calidad de las representaciones TF-IDF y los embeddings.

Paso 2 — Normalización de consultas SQL

Se utilizó la librería `sqlparse` con configuración: `keyword_case='upper'`, `identifier_case='lower'`, `strip_comments=True`. Esta normalización garantiza que dos consultas SQL semánticamente equivalentes con diferente capitalización sean tratadas como idénticas.

Paso 3 — Eliminación de duplicados exactos

Se eliminaron los 33 pares (`pregunta_limpia`, `SQL_limpio`) exactamente idénticos detectados en el EDA. Los duplicados con SQL idéntica, pero pregunta diferente (paráfrasis) se conservaron intencionalmente, pues representan variabilidad lingüística valiosa para el entrenamiento.

Paso 4 — Filtro de outliers y validación

Se filtró el conjunto eliminando preguntas de menos de 3 palabras y consultas SQL sin cláusula `SELECT`. En el dataset Spider ambos filtros resultaron en 0 eliminaciones adicionales, confirmando la alta calidad base.

3.2 Resultados de la Limpieza

Métrica	Antes	Después	Diferencia
N registros totales	8,034	8,001	-33
Media palabras (NL)	12.69	12.71	+0.02
Media tokens (SQL)	17.8	17.8	0.00
Bases de datos únicas	200	200	0
Duplicados exactos	33	0	-33

El pipeline de limpieza preservó el 99.6% de los datos originales (8,001 de 8,034 registros), lo cual confirma que Spider es un dataset de alta calidad que no requiere limpieza agresiva. La pérdida de datos es mínima y metodológicamente justificada.

4. Feature Engineering Avanzado

El Feature Engineering transforma los datos textuales en representaciones numéricas que los modelos de aprendizaje automático pueden procesar. Para Text-to-SQL se construyeron tres familias de features, resultando en un total de 55 variables por ejemplo.

4.1 Features Lingüísticas de la Pregunta NL (16 features)

Se extrajo un conjunto de características sobre la estructura y contenido semántico de las preguntas en lenguaje natural:

Feature	Descripción	Relevancia para T2SQL
nl_word_count	Número de palabras	Proxy de complejidad de la intención
nl_lexical_diversity	Ratio palabras únicas/total	Indicador de especificidad semántica
nl_has_wh_word	Presencia de what/which/who/where	Tipo de consulta esperado

Feature	Descripción	Relevancia para T2SQL
nl_has_agg_word	Palabras: many/average/total/count	Predice GROUP BY o funciones de agregación
nl_has_comparison_word	Palabras: more/less/between/not	Predice HAVING o condiciones complejas
nl_has_order_word	Palabras: top/rank/sort/order	Predice ORDER BY o LIMIT
nl_has_join_signal	Palabras: across/between/related	Predice necesidad de JOIN
nl_has_number	Presencia de valores numéricos	Predice constantes en WHERE o LIMIT

4.2 Features Estructurales de SQL (17 features)

Se construyeron características que capturan la estructura interna de las consultas SQL objetivo:

- **sql_nesting_depth**: Profundidad de anidamiento de subconsultas (número de SELECT adicionales).
- **sql_tables_referenced**: Número de tablas únicas referenciadas en FROM y JOIN.
- **sql_where_conditions**: Conteo de condiciones en la cláusula WHERE (separadas por AND/OR).
- **sql_total_keywords**: Número total de palabras clave SQL presentes en la consulta.
- **sql_join_type_***: Indicadores binarios para tipo de JOIN (none/inner/left/right).
- **sql_has_in_clause / sql_has_like / sql_has_null_check / sql_has_between**: Predicados de filtro especiales.

4.3 Features de Interacción NL-SQL (3 features)

Se crearon features que capturan la relación semántica entre la pregunta NL y la consulta SQL esperada:

- **ratio_nl_sql_length**: Razón entre longitud NL y longitud SQL — valores bajos indican preguntas concisas con SQL complejas.
- **interaction_score**: Score compuesto que mide la coherencia semántica NL-SQL. Ejemplo: si la pregunta contiene 'average' y el SQL tiene GROUP BY, la coherencia es alta.

- `sql_complexity_per_word`: Complejidad SQL normalizada por número de palabras en la pregunta — mide la densidad informacional.

4.4 Score de Complejidad SQL Compuesto

Se diseñó un score de complejidad propio que combina múltiples indicadores con pesos diferenciados según su impacto en la dificultad real:

Componente	Peso	Justificación
<code>has_subquery</code>	3.0	Subconsultas requieren razonamiento anidado
<code>has_union</code> / <code>has_intersect</code> / <code>has_except</code>	2.5 c/u	Operadores de conjunto son raros y complejos
<code>has_join</code>	2.0	JOIN requiere comprensión de relaciones FK
<code>has_having</code>	2.0	HAVING implica filtrado post-agregación
<code>has_group_by</code>	1.5	GROUP BY introduce semántica de agrupación
<code>has_distinct</code>	1.0	Deduplica resultados — semántica adicional
<code>join_count</code>	$1.0 \times n$	Múltiples JOINS escalan la complejidad

4.5 Análisis de Importancia de Features

Se entrenó un Random Forest con 100 estimadores para estimar la importancia relativa de las features como proxy de su poder predictivo para la variable objetivo (dificultad):

Ranking	Feature	Importancia
1°	<code>complexity_score</code>	20.33%
2°	<code>sql_complexity_per_word</code>	8.66%
3°	<code>has_order_by</code>	8.54%
4°	<code>sql_token_count</code>	8.33%
5°	<code>has_subquery</code>	7.55%

Ranking	Feature	Importancia
6°	sql_nesting_depth	~6.8%
7°	join_count	~5.9%
8°	has_group_by	~5.1%
9°	nl_has_agg_word	~4.2%
10°	clause_count	~3.8%

El score de complejidad diseñado ad hoc captura el 20.33% de la varianza predictiva — es la feature más importante, superando a métricas directas como `sql_token_count`. Esto valida el diseño del score compuesto como aporte metodológico propio.

5. Estrategias de Balanceamiento de Datos

El desbalance detectado en el EDA (ratio 5.9x entre medium y extra) fue abordado mediante tres estrategias complementarias. Cada estrategia opera sobre el split de train únicamente para evitar contaminación del set de evaluación.

Clase	Registros Train	Porcentaje	Peso Asignado
Easy (0)	2,024	29.0%	0.8608
Medium (1)	3,472	49.8%	0.5018
Hard (2)	896	12.9%	1.9445
Extra (3)	577	8.3%	3.0195
TOTAL	6,969	100%	—

5.1 Estrategia 1 — Class Weights (Recomendada para LLMs)

Se calcularon pesos inversamente proporcionales a la frecuencia de cada clase usando `compute_class_weight('balanced')` de `scikit-learn`. Los pesos resultantes compensan el desbalance sin modificar los datos de entrenamiento.

- Ventaja principal: no genera datos artificiales — preserva la distribución real del fenómeno.
- Ideal para: fine-tuning de modelos LLM (CodeLlama-13B, LLaMA-3-8B con QLoRA) donde la función de pérdida admite pesos por clase.
- Limitación: requiere soporte explícito en el framework de entrenamiento (PyTorch, HuggingFace Trainer).

5.2 Estrategia 2 — SMOTE

Synthetic Minority Over-sampling Technique genera ejemplos sintéticos de las clases minoritarias mediante interpolación en el espacio de features numéricas. Con `k_neighbors=5` se generaron 6,919 ejemplos sintéticos.

Clase	Original	Post-SMOTE	Sintéticos
Easy	2,024	3,472	+1,448
Medium	3,472	3,472	0
Hard	896	3,472	+2,576
Extra	577	3,472	+2,895
TOTAL	6,969	13,888	+6,919

SMOTE opera sobre features numéricas. Los ejemplos sintéticos no generan texto NL nuevo — son interpolaciones en el espacio de features. Usar con cautela en contextos donde el texto original es necesario para entrenamiento.

5.3 Estrategia 3 — SMOTETomek (Combinada)

SMOTETomek combina SMOTE con el método Tomek Links, que elimina ejemplos ambiguos en las fronteras de decisión entre clases. El resultado es un dataset balanceado y con menor ruido en las zonas de frontera.

- Registros finales: 13,886 (vs. 13,888 de SMOTE puro).
- La diferencia de 2 registros corresponde a ejemplos Tomek eliminados por ambigüedad.
- Recomendada cuando se trabaja con clasificadores de frontera dura (SVM, KNN).

6. Técnicas de Data Augmentation para Text-to-SQL

Para Text-to-SQL, el data augmentation presenta un desafío único: cualquier transformación sobre la pregunta NL o la consulta SQL debe preservar la correspondencia semántica entre ambas. Se implementaron dos técnicas sobre las clases minoritarias (hard y extra).

6.1 Augmentation de Preguntas NL — Paráfrasis Lexical

Se desarrolló un sistema de sustitución de patrones lingüísticos específicos para Text-to-SQL, basado en un diccionario de 14 categorías de paráfrasis. El diccionario cubre los principales patrones de pregunta presentes en Spider:

Patrón Original	Variantes Generadas	Tipo
How many	What is the count of / What is the number of	Interrogativo-cuantitativo
What are	List all / Show all / Find all / Display all	Listado
What is	Show me the / Find the / Tell me the	Consulta directa
the average	the mean / the avg	Agregación
the maximum	the highest / the largest / the greatest	Superlativo
the minimum	the lowest / the smallest / the least	Superlativo
Show me	Find / List / Display / Retrieve	Imperativo

Cada augmentation aplica una sola sustitución por ejemplo para evitar cambios de significado acumulativos. La técnica es conservadora por diseño — solo sustituye patrones con semántica demostrada como equivalente en el contexto Text-to-SQL.

6.2 Augmentation de Consultas SQL — Variantes Equivalentes

Se generaron variantes SQL semánticamente equivalentes mediante transformaciones formalmente verificables:

- JOIN → INNER JOIN (equivalentes en SQL estándar cuando no se especifica tipo).

- ORDER BY sin ASC/DESC → ORDER BY ... ASC (comportamiento por defecto explícito).
- Conservación de subconsultas complejas sin modificación para evitar errores de equivalencia.

6.3 Resultados de la Augmentation

Clase	Original (Train)	Post-Augmentation	Ratio de Aumento
Easy	2,024	2,024	0% (clase mayoritaria)
Medium	3,472	3,472	0% (clase mayoritaria)
Hard	896	2,272	+154% (+1,376)
Extra	577	1,353	+135% (+776)
TOTAL	6,969	9,197	+32%

Se generaron 2,228 ejemplos nuevos (151% de aumento sobre las clases minoritarias), reduciendo el ratio de desbalance de 5.9x original a aproximadamente 2.4x post-augmentation, nivel considerado manejable con class weights adicionales.

7. Pipeline de Preprocesamiento Automatizado

Todos los pasos anteriores fueron encapsulados en un pipeline reproducible y serializable, implementado bajo el estándar de scikit-learn Pipeline y ColumnTransformer. Este diseño garantiza que nuevos datos puedan ser procesados aplicando exactamente las mismas transformaciones aprendidas sobre el conjunto de train.

7.1 Arquitectura del Pipeline

El pipeline opera en dos ramas paralelas que luego se concatenan:

Rama	Entrada	Transformaciones	Salida
Numérica	29 features numéricas	SimpleImputer(median) → RobustScaler()	29 features escaladas
Textual	Pregunta NL (string)	TF-IDF(500, ngram=(1,2)) → PCA(n=50)	50 componentes principales
Final	Concatenación	—	79 features por ejemplo

Se optó por RobustScaler en lugar del StandardScaler convencional dado que el score de complejidad y el conteo de JOINS presentan distribuciones asimétricas con outliers. RobustScaler usa la mediana y el rango intercuartílico, siendo resistente a valores extremos.

La reducción PCA sobre los vectores TF-IDF comprime 500 dimensiones en 50 componentes principales, capturando la mayor parte de la varianza lingüística mientras reduce el riesgo de sobreajuste.

7.2 Resultados de Validación — Modelo Baseline

Se entrenó un modelo Random Forest (200 estimadores, max_depth=12, class_weight='balanced') sobre el pipeline completo para validar que las transformaciones producen features útiles para la tarea:

Accuracy en conjunto de validación				99.42%
Clase	Precision	Recall	F1-Score	Support
Easy	1.00	1.00	1.00	331
Medium	1.00	1.00	1.00	567
Hard	0.98	0.99	0.98	103
Extra	0.99	0.97	0.98	31
Macro avg	0.99	0.99	0.99	1,032

El 99.42% de accuracy indica que las features construidas capturan casi perfectamente la lógica de dificultad del dataset Spider. Este resultado es esperado dado que la dificultad en Spider está definida por reglas estructurales explícitas sobre el SQL — las features SQL capturan exactamente esas reglas.

7.3 Artefactos Generados para Producción

El pipeline fue serializado con joblib, generando los siguientes artefactos listos para despliegue o reutilización:

- preprocessor.joblib: Pipeline completo de transformación (imputer + scaler + TF-IDF + PCA).
- baseline_model.joblib: Modelo Random Forest entrenado, listo para inferencia.
- label_encoder_db.joblib: Encoder para el campo db_id (200 bases de datos).
- difficulty_map.joblib: Mapeo bidireccional de etiquetas de dificultad.
- spider_preprocessed.csv: Dataset completo procesado con todas las features en formato CSV.

7.4 Función de Inferencia en Producción

Se implementó la función predict_difficulty(question, sql) que acepta una nueva pregunta y consulta SQL, aplica el pipeline completo y retorna la predicción de dificultad con probabilidades por clase. Ejemplos de validación:

Pregunta	SQL	Predicción	Confianza
How many singers are there?	SELECT count(*) FROM singer	EASY	99.89%
What is the average age of singers from USA?	SELECT avg(age) FROM singer WHERE...	MEDIUM	~97%
List all concert names and their stadiums	SELECT ... FROM concert JOIN stadium ...	MEDIUM	~95%
Names of concerts with >10000 attendees and stadiums capacity >50000	SELECT ... JOIN ... WHERE T1.att > 10000 AND T2.cap > 50000	HARD	~91%

8. Resumen

La siguiente tabla resume los resultados cuantitativos de cada etapa del pipeline de preparación de datos:

Etapa	Resultado Principal
Dataset (carga completa)	8,034 ejemplos — 200 DBs — 138 dominios — 4 niveles de dificultad
EDA — Desbalance detectado	Ratio 5.9x (medium: 49% vs extra: 8.3%) — requiere balanceamiento
EDA — Calidad	0 valores nulos — 33 duplicados exactos — alta calidad base
Limpieza	99.6% integridad conservada (8,001 registros finales)
Feature Engineering	55 features totales: 16 lingüísticas + 17 SQL + 3 interacción + 19 base
Feature más importante	complexity_score (diseño propio): 20.33% de importancia predictiva
Balanceamiento — Class Weights	Pesos: easy=0.86 / medium=0.50 / hard=1.94 / extra=3.02
Balanceamiento — SMOTE	13,888 registros post-resampling (6,919 sintéticos generados)
Data Augmentation	2,228 nuevos ejemplos — ratio desbalance reducido de 5.9x a 2.4x
Pipeline output	79 features por ejemplo (29 numéricas + 50 PCA-TF-IDF)
Baseline accuracy	99.42% — valida la calidad de las features construidas

Decisiones Técnicas Justificadas

- RobustScaler sobre StandardScaler: distribuciones asimétricas en features SQL con outliers significativos.
- PCA(50) sobre TF-IDF: balance entre información lingüística y riesgo de sobreajuste (500→50 dims).
- Augmentation solo en clases minoritarias: evita amplificar el desbalance existente.

- Class Weights como estrategia recomendada para el modelo final (LLM/Transformer): no altera la distribución real de los datos de entrenamiento, que es la recomendación dominante en la literatura reciente de fine-tuning de LLMs.

Próximos Pasos

- Fase de modelado: fine-tuning de CodeLlama-13B y LLaMA-3-8B con QLoRA sobre el dataset procesado.
- Evaluación: Execution Accuracy (EX) sobre el test set oficial de Spider.
- Comparativa: análisis diferencial con BIRD (datos ruidosos reales) y MultiSpider en español.

Referencias

- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. Proceedings of EMNLP 2018. <https://arxiv.org/abs/1809.08887>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. <https://arxiv.org/abs/2106.09685>
- Dou, L. et al. (2022). MultiSpider: Towards Benchmarking Multilingual Text-to-SQL Semantic Parsing. AAAI 2023. <https://arxiv.org/abs/2212.13492>
- Li, J. et al. (2023). Can LLM Already Serve as a Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQL. NeurIPS 2024. <https://bird-bench.github.io>